

UC San Diego

Electrical and Computer Engineering
JACOBS SCHOOL OF ENGINEERING



Score-Based Diffusion Models

ECE 285 Deep Generative Models

Youwei Liang

Today's plan

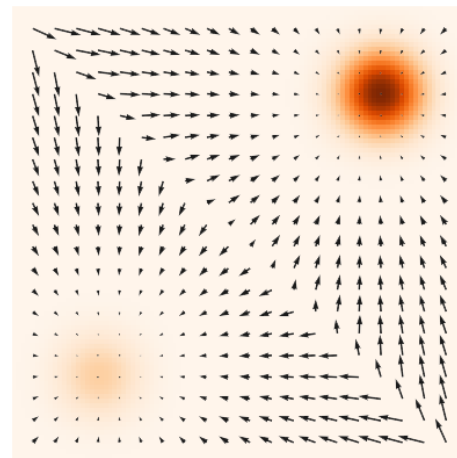
- Score matching and sliced score matching
- Sampling with Langevin dynamics
- Noise Conditional Score Networks
- Score-based diffusion models

Score matching

- Score function
 - Gradient of the log probability density function of data

Score function: $f(\mathbf{x}) = \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$.

Follow the score to reach a data sample!



pdf and score

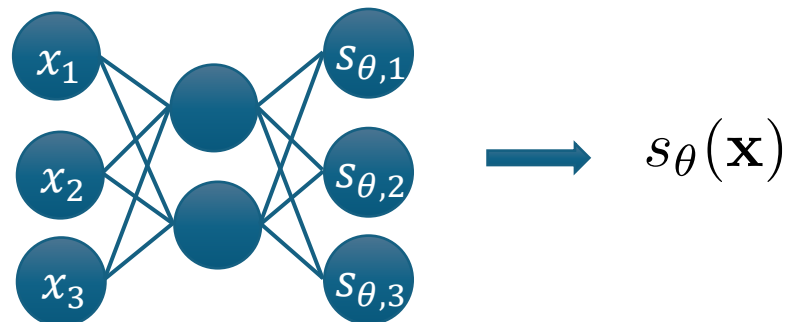
- Score matching
 - Minimize the Fisher Divergence between the score function and the score network

$$\arg \min_{\theta} \frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\|\mathbf{s}_{\theta}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})\|_2^2]$$

$$\arg \min_{\theta} \frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \left[\text{tr}(\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x})) + \frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x})\|_2^2 \right]$$

Score matching

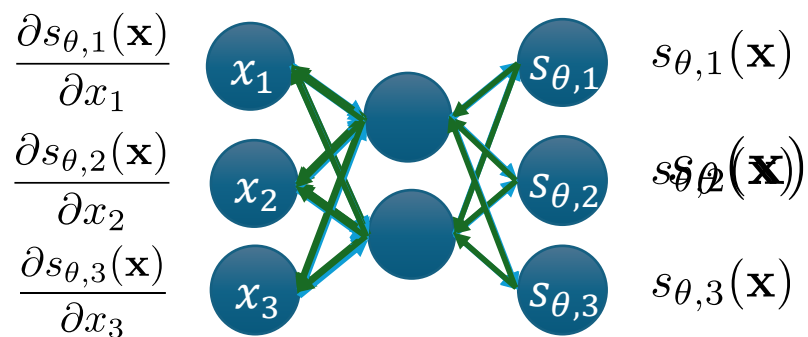
- Deep neural networks as more expressive score models



Objective

$$\arg \min_{\theta} \frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \left[\text{tr}(\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x})) + \frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x})\|_2^2 \right]$$

Compute $\text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}))$



$O(d)$ Backprop

$$\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) = \begin{pmatrix} \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_3} \\ \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_3} \\ \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_3} \end{pmatrix}$$

Score matching

```
import torch
import torch.nn as nn

# ---- Simple MLP score function: s_theta(x) in R^D ----
class ScoreMLP(nn.Module):
    def __init__(self, dim, hidden=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(dim, hidden),
            nn.Tanh(),
            nn.Linear(hidden, hidden),
            nn.Tanh(),
            nn.Linear(hidden, dim),
        )

    def forward(self, x):
        return self.net(x)
```

```
# ---- demo usage ----
B, D = 32, 50
x = torch.randn(B, D)

model = ScoreMLP(dim=D, hidden=128)
loss = score_matching_objective(model, x)

loss.backward()
print("loss:", float(loss))
```

```
def score_matching_objective(model, x):
    """
    x: (B, D) minibatch of data
    returns: scalar objective (averaged over batch)
    """
    # We need gradients w.r.t. x to compute  $\text{tr}(\nabla_x s(x))$ 
    x = x.detach().requires_grad_(True) # (B, D)

    s = model(x) # (B, D)

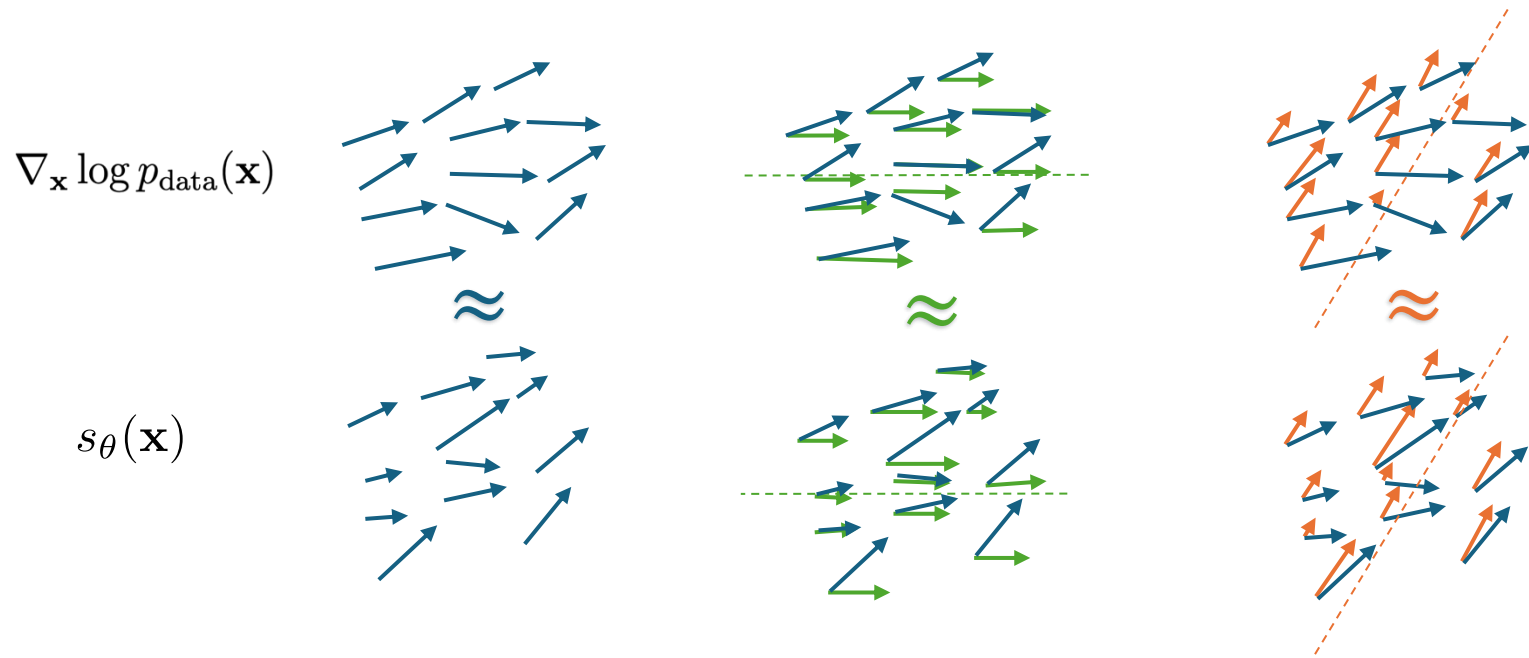
    #  $0.5 * ||s(x)||^2$  term (per-sample), then average
    sq_norm = (s ** 2).sum(dim=1) # (B,)

    # divergence term:  $\text{tr}(\nabla_x s(x)) = \sum_i \partial s_i / \partial x_i$ 
    # Compute by summing diagonal Jacobian entries, one dim at a time.
    div = 0.0
    D = x.shape[1]
    for i in range(D):
        # gradient of sum of  $s[:, i]$  over batch w.r.t  $x \rightarrow (B, D)$ 
        grad_si = torch.autograd.grad(
            outputs=s[:, i].sum(),
            inputs=x,
            create_graph=True,
            retain_graph=True,
            only_inputs=True,
        )[0]
        div = div + grad_si[:, i] # take diagonal entry (B,)

    # Full objective per sample:  $0.5 * ( \text{div} + 0.5 * ||s||^2 )$ 
    loss_per_sample = 0.5 * (div + 0.5 * sq_norm)
    return loss_per_sample.mean()
```

Sliced score matching

- Consider projection of score vectors onto random directions



Objective: sliced Fisher divergence

$$\frac{1}{2} E_{\mathbf{v} \sim p_{\mathbf{v}}} E_{\mathbf{x} \sim p_{\text{data}}} [(\mathbf{v}^{\top} \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \mathbf{v}^{\top} \mathbf{s}_{\theta}(\mathbf{x}))^2]$$

Sliced score matching

- Objective: sliced Fisher divergence

$$\begin{aligned} L(\boldsymbol{\theta}; p_v) &= \frac{1}{2} \mathbb{E}_{p_v} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [(\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}) - \mathbf{v}^T \mathbf{s}_{\text{data}}(\mathbf{x}))^2] \\ &= \frac{1}{2} \mathbb{E}_{p_v} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [(\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}))^2 + (\mathbf{v}^T \mathbf{s}_{\text{data}}(\mathbf{x}))^2 - 2(\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}))(\mathbf{v}^T \mathbf{s}_{\text{data}}(\mathbf{x}))] \\ &= \mathbb{E}_{p_v} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \left[\frac{1}{2} (\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}))^2 - (\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}))(\mathbf{v}^T \mathbf{s}_{\text{data}}(\mathbf{x})) \right] + C \\ &\quad - \mathbb{E}_{p_v} \mathbb{E}_{p_{\text{data}}} [(\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}))(\mathbf{v}^T \mathbf{s}_{\text{data}}(\mathbf{x}))] \\ &= - \mathbb{E}_{p_v} \int [(\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}))(\mathbf{v}^T \mathbf{s}_{\text{data}}(\mathbf{x})) p_{\text{data}} d\mathbf{x}] \\ &= - \mathbb{E}_{p_v} \left[\int (\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x})) (\mathbf{v}^T \nabla_{\mathbf{x}} \log p_{\text{data}}) p_{\text{data}} d\mathbf{x} \right] \\ &= - \mathbb{E}_{p_v} \left[\int (\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x})) (\mathbf{v}^T \nabla_{\mathbf{x}} p_{\text{data}}) d\mathbf{x} \right] \\ &= - \mathbb{E}_{p_v} \left[\int (\mathbf{v}^T \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x})) (\mathbf{v}^T \nabla_{\mathbf{x}} p_{\text{data}}) d\mathbf{x} \right] \\ &= \mathbb{E}_{p_v} \left[\int \mathbf{v}^T \nabla_{\mathbf{x}} \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}) \mathbf{v} \cdot p_{\text{data}} d\mathbf{x} \right] \\ &= \mathbb{E}_{p_v} \mathbb{E}_{p_{\text{data}}} [\mathbf{v}^T \nabla_{\mathbf{x}} \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}) \mathbf{v}] \end{aligned}$$

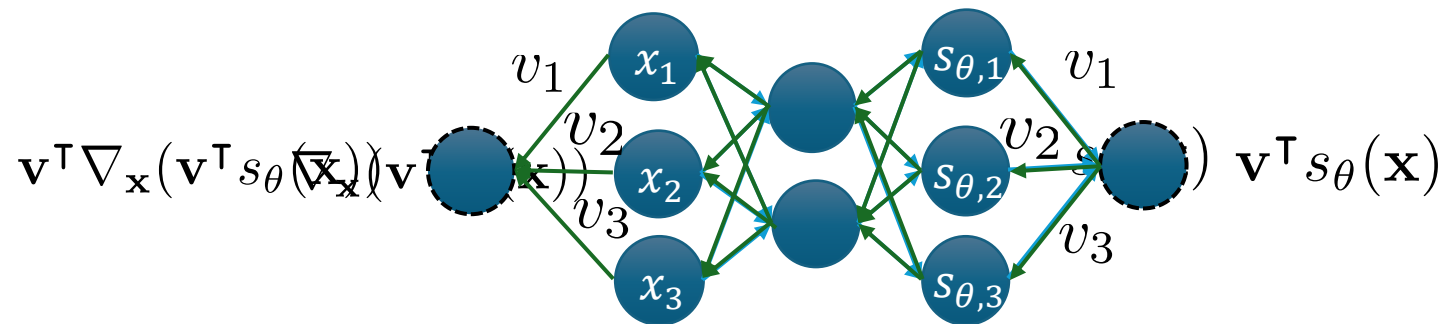
Sliced score matching

$$E_{\mathbf{v} \sim p_{\mathbf{v}}} E_{\mathbf{x} \sim p_{\text{data}}} \left[\mathbf{v}^\top \nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) \mathbf{v} + \frac{1}{2} (\mathbf{v}^\top s_{\theta}(\mathbf{x}))^2 \right]$$

$$\begin{pmatrix} v_1 & v_2 & v_3 \end{pmatrix} \begin{pmatrix} \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_3} \\ \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_3} \\ \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_3} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

$$\mathbf{v}^\top \nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) \mathbf{v} = \boxed{\mathbf{v}^\top \nabla_{\mathbf{x}} (\mathbf{v}^\top s_{\theta}(\mathbf{x}))}$$

No Jacobian needed



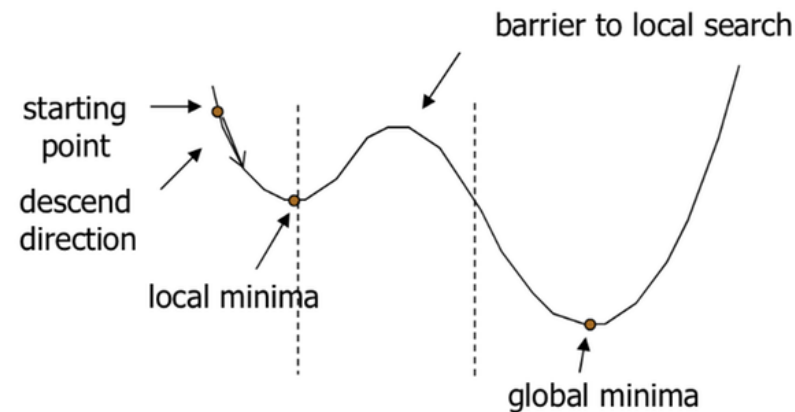
Sampling with Langevin dynamics

- Langevin dynamics
 - Markov Chain Monte Carlo (MCMC) method

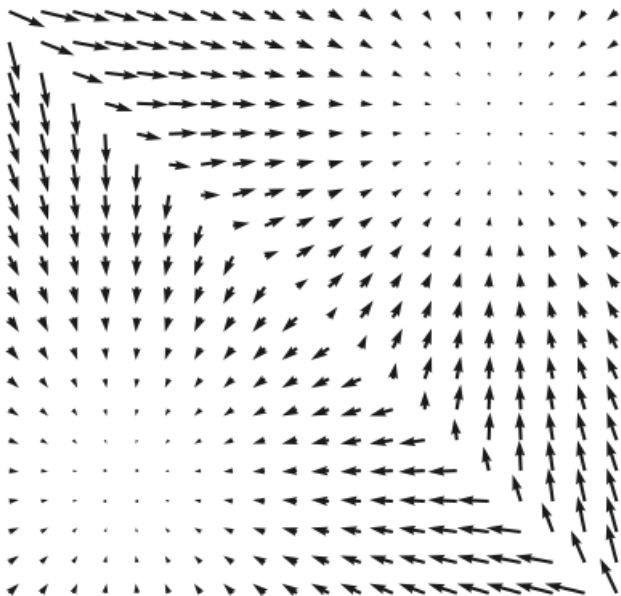
Start from a random point $\mathbf{x}_0 \sim \boldsymbol{\pi}(\mathbf{x})$

$$\tilde{\mathbf{x}}_t = \tilde{\mathbf{x}}_{t-1} + \frac{\epsilon}{2} \nabla_{\mathbf{x}} \log p(\tilde{\mathbf{x}}_{t-1}) + \sqrt{\epsilon} \mathbf{z}_t \quad \mathbf{z}_t \sim \mathcal{N}(0, I)$$

As $\epsilon \rightarrow 0$ and $T \rightarrow \infty$, $\tilde{\mathbf{x}}_T$ converges to $p_{\text{data}}(\mathbf{x})$

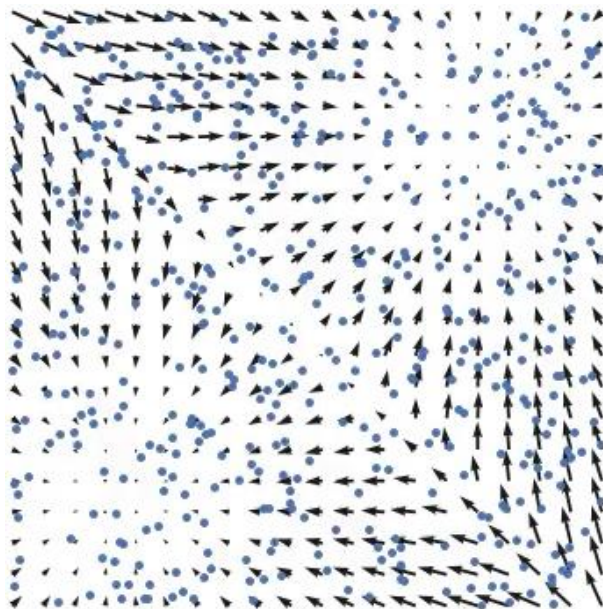


Langevin dynamics



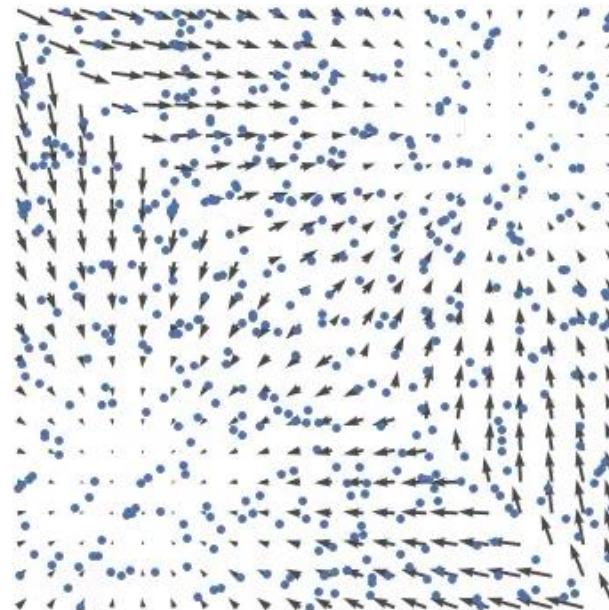
Scores

$$\mathbf{s}_\theta(\mathbf{x})$$



Follow the scores

$$\tilde{\mathbf{x}}_{t+1} \leftarrow \tilde{\mathbf{x}}_t + \frac{\epsilon}{2} \mathbf{s}_\theta(\tilde{\mathbf{x}}_t)$$



Follow noisy scores:
Langevin MCMC

$$\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$
$$\tilde{\mathbf{x}}_{t+1} \leftarrow \tilde{\mathbf{x}}_t + \frac{\epsilon}{2} \mathbf{s}_\theta(\tilde{\mathbf{x}}_t) + \sqrt{\epsilon} \mathbf{z}_t$$

Sliced score matching

```
def sliced_score_matching_objective(model, x, K=1):
    x = x.detach().requires_grad_(True) # need grads wrt x
    s = model(x) # (B, D)
    B, D = x.shape

    # 0.5 * ||s(x)||^2 term (per sample)
    sq_norm = (s ** 2).sum(dim=1) # (B,)

    # Estimate divergence via: tr(J) = E_v[ v^T J v ] where J = ∇_x s(x)
    # v^T J v = sum_i v_i * ∂(s·v)/∂x_i = <v, ∇_x (s·v)>
    div_est = 0.0
    for _ in range(K):
        v = torch.randn_like(x)

        # scalar per sample: (s · v)
        sv = (s * v).sum(dim=1) # (B,)

        # gradient wrt x: ∇_x (s·v) -> (B, D)
        grad_sv = torch.autograd.grad(
            outputs=sv.sum(), inputs=x,
            create_graph=True, retain_graph=True, only_inputs=True,
        )[0]

        # v^T ∇_x (s·v) -> (B,)
        vJv = (grad_sv * v).sum(dim=1)
        div_est = div_est + vJv

    div_est = div_est / K # average over projections

    # SSM objective
    # 0.5 * (div_est + 0.5*||s||^2)
    loss_per_sample = 0.5 * (div_est + 0.5 * sq_norm)
    return loss_per_sample.mean()
```

```
@torch.no_grad()
def langevin_dynamics_sample(
    score_model,
    x_init,
    n_steps=200,
    step_size=1e-3,
    noise_scale=None,
    clamp=None,
):
    """
    Unadjusted Langevin Algorithm (ULA):
    | x_{t+1} = x_t + step_size * s_theta(x_t) + sqrt(2*step_size) * z_t

    Args:
    | score_model: nn.Module mapping (B,D)->(B,D)
    | x_init: (B,D) initial noise samples
    | n_steps: number of Langevin steps
    | step_size: epsilon
    | noise_scale: if None, uses sqrt(2*step_size); else overrides noise std
    | clamp: optional tuple (low, high) to clamp samples each step (for demo stability)

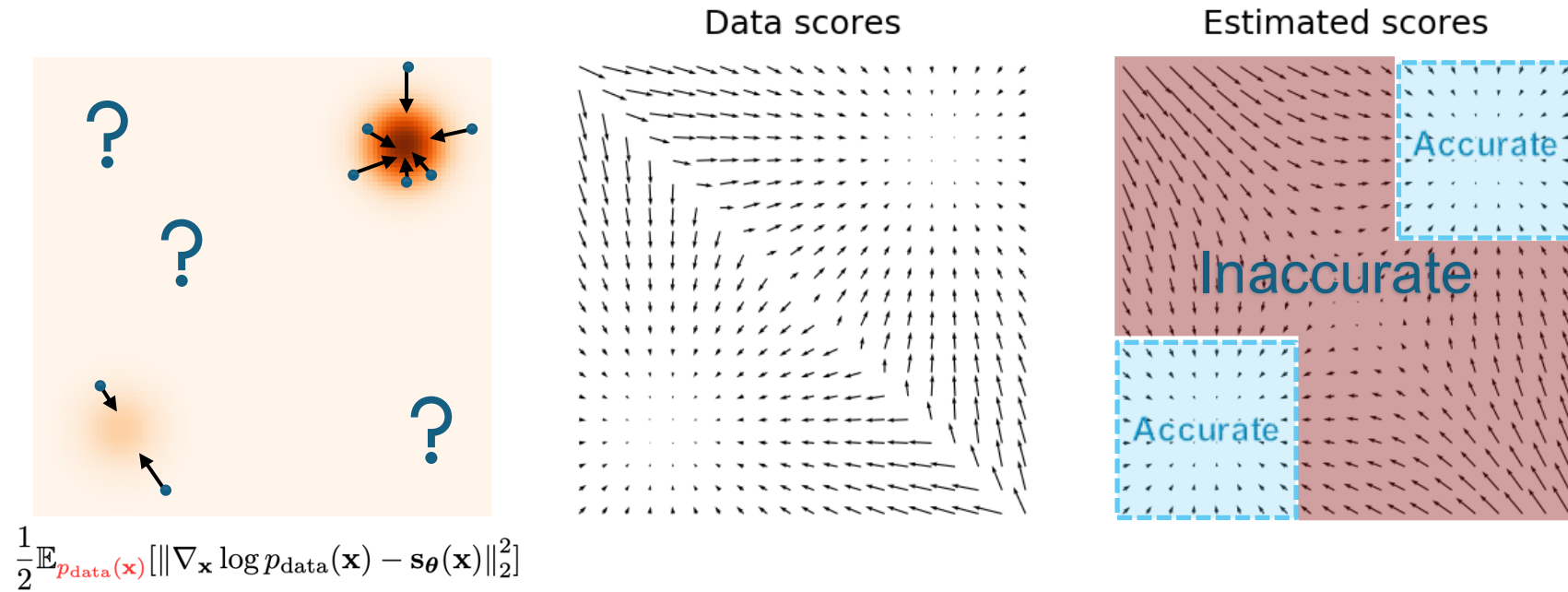
    Returns:
    | x: (B,D) samples
    """
    x = x_init.clone()

    if noise_scale is None:
        noise_scale = (2.0 * step_size) ** 0.5 # sqrt(2ε)

    for _ in range(n_steps):
        s = score_model(x) # score field, (B,D)
        z = torch.randn_like(x)
        x = x + step_size * s + noise_scale * z
        if clamp is not None:
            x = x.clamp(clamp[0], clamp[1])

    return x
```

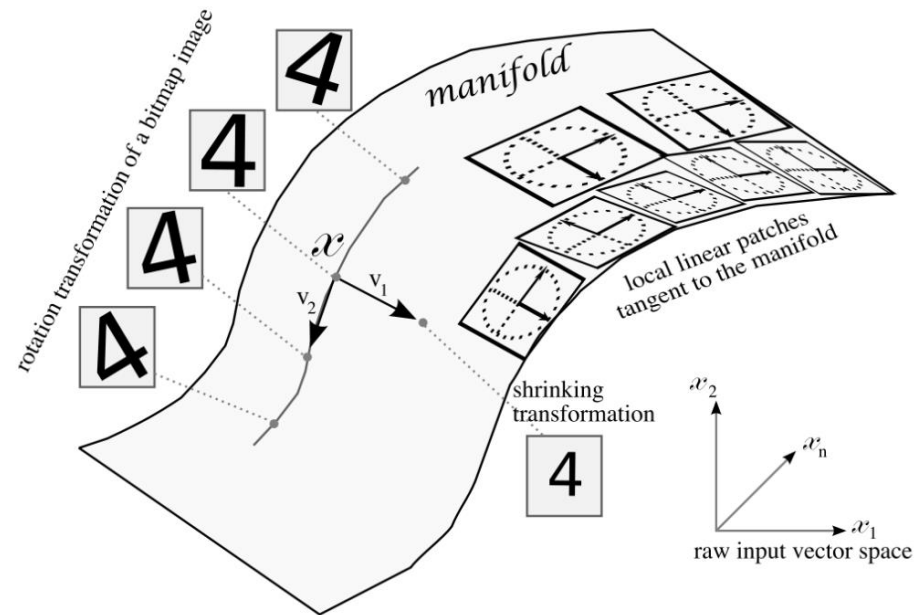
Challenge in low data density regions



Score function is inaccurate at low data regions

The Manifold Hypothesis

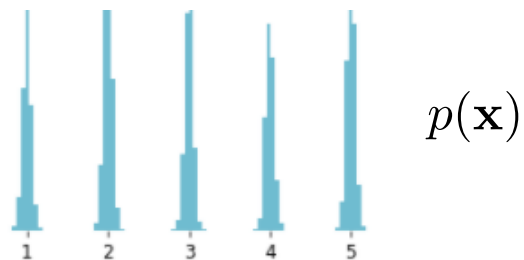
- Real-world data often lies in a low-dimensional manifold embedded in a high-dimensional space
- “Empty” space without any data
 - Score function is undefined (or simply 0)



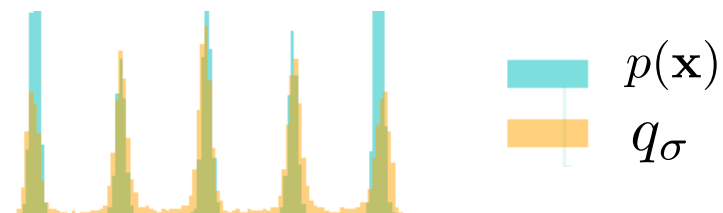
Denoising score matching

- Add Gaussian noise to data
 - Expand the data to filled the whole space

$$q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) = \mathcal{N}(\mathbf{x}; \sigma^2 I)$$



$$q_{\sigma}(\tilde{\mathbf{x}}) = \int p(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) d\mathbf{x}$$



$$p_{\text{data}}(\mathbf{x}) \longrightarrow q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) \longrightarrow q_{\sigma}(\tilde{\mathbf{x}})$$



$\mathbf{x}$



$\tilde{\mathbf{x}}$

Denoising score matching

$$\begin{aligned} & \frac{1}{2} E_{\tilde{\mathbf{x}} \sim p_{\text{data}}} [\| \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}) \|_2^2] \\ &= \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\| \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) \|_2^2] + \text{const.} \end{aligned}$$

$$q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}} | \mathbf{x}, \sigma^2 \mathbf{I})$$

$$\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) = -\frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2}$$

Training objective

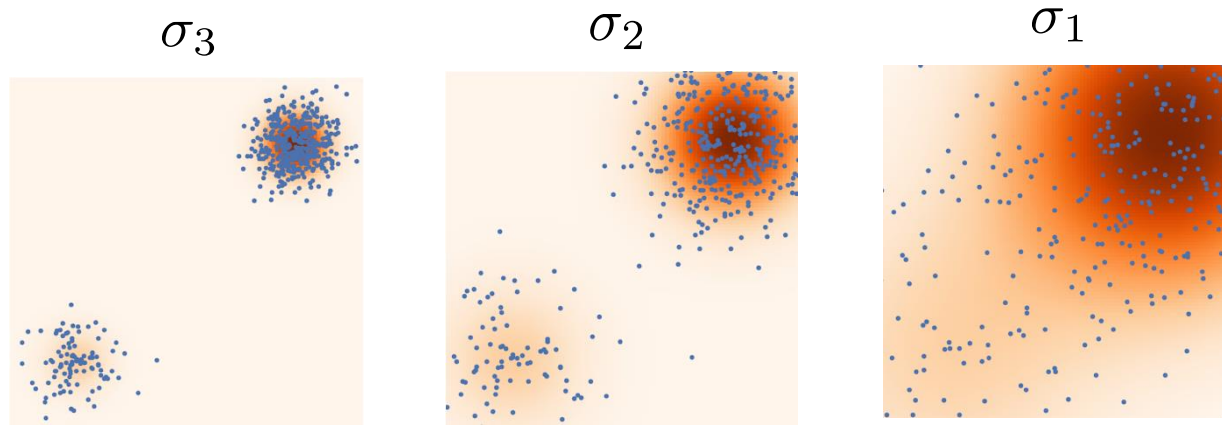
$$\frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}, \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \mathbf{s}_{\theta}(\mathbf{x} + \sigma \mathbf{z}) + \frac{\mathbf{z}}{\sigma} \right\|_2^2 \right]$$

Normalization

Noise Conditional Score Networks

- Adding noise at different noise level

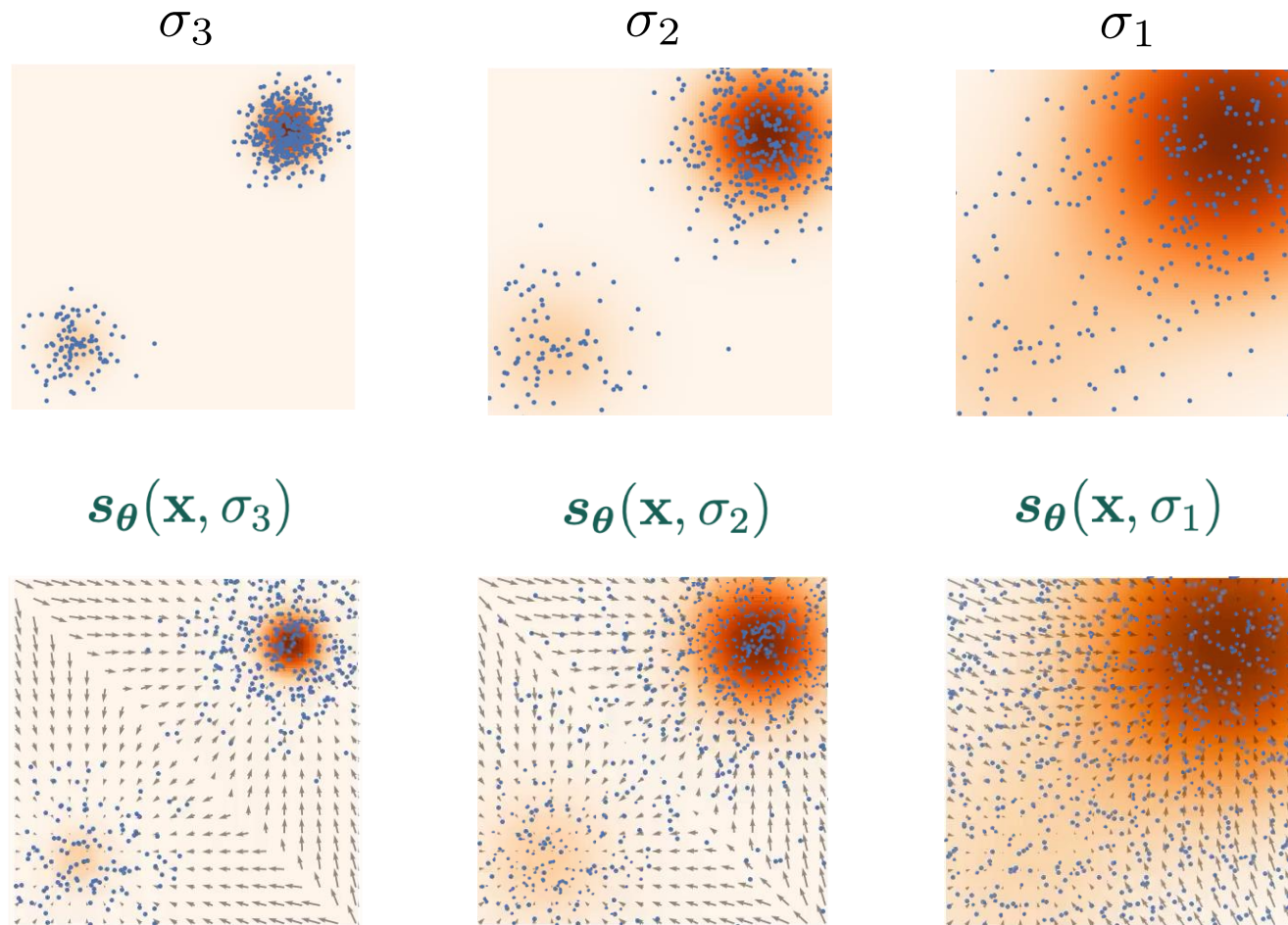
$$\frac{\sigma_1}{\sigma_2} = \dots = \frac{\sigma_{L-1}}{\sigma_L} > 1$$



$$\ell(\boldsymbol{\theta}; \sigma) \triangleq \frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{\tilde{\mathbf{x}} \sim \mathcal{N}(\mathbf{x}, \sigma^2 I)} \left[\left\| \mathbf{s}_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}, \sigma) + \frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2} \right\|_2^2 \right]$$

$$\mathcal{L}(\boldsymbol{\theta}; \{\sigma_i\}_{i=1}^L) \triangleq \frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) \ell(\boldsymbol{\theta}; \sigma_i) \quad \lambda(\sigma) = \sigma^2$$

Sampling with annealed Langevin dynamics



Algorithm 1 Annealed Langevin dynamics.

Require: $\{\sigma_i\}_{i=1}^L, \epsilon, T$.

- 1: Initialize $\tilde{\mathbf{x}}_0$
- 2: **for** $i \leftarrow 1$ to L **do**
- 3: $\alpha_i \leftarrow \epsilon \cdot \sigma_i^2 / \sigma_L^2$ $\triangleright \alpha_i$ is the step size.
- 4: **for** $t \leftarrow 1$ to T **do**
- 5: Draw $\mathbf{z}_t \sim \mathcal{N}(0, I)$
- 6: $\tilde{\mathbf{x}}_t \leftarrow \tilde{\mathbf{x}}_{t-1} + \frac{\alpha_i}{2} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}_{t-1}, \sigma_i) + \sqrt{\alpha_i} \mathbf{z}_t$
- 7: **end for**
- 8: $\tilde{\mathbf{x}}_0 \leftarrow \tilde{\mathbf{x}}_T$
- 9: **end for**
- return** $\tilde{\mathbf{x}}_T$

$$\alpha_i = \epsilon \cdot \sigma_i^2 / \sigma_L^2$$

Noise Conditional Score Networks

```
class NCSNScoreMLP(nn.Module):
    def __init__(self, dim, hidden=128, n_sigmas=10):
        super().__init__()
        self.dim = dim
        self.sigma_embed = nn.Embedding(n_sigmas, hidden)
        self.net = nn.Sequential(
            nn.Linear(dim + hidden, hidden),
            nn.SiLU(),
            nn.Linear(hidden, hidden),
            nn.SiLU(),
            nn.Linear(hidden, dim),
        )

    def forward(self, x_tilde, sigma_idx):
        """
        x_tilde: (B, D) noisy inputs
        sigma_idx: (B,) integer indices selecting a noise level
        """
        emb = self.sigma_embed(sigma_idx)  # (B, hidden)
        h = torch.cat([x_tilde, emb], dim=1)  # (B, D+hidden)
        return self.net(h)  # (B, D)
```

```
B, D = 64, 2
x = torch.randn(B, D)

sigmas = torch.tensor([1.0, 0.5, 0.25, 0.125]) # noise levels (high -> low)
model = NCSNScoreMLP(dim=D, hidden=128, n_sigmas=len(sigmas))

loss = dsm_ncsn_loss(model, x, sigmas, weight_by_sigma2=True)
loss.backward()
```

```
def dsm_ncsn_loss(score_model, x, sigmas, weight_by_sigma2=True):
    """
    x:      (B, D) clean data minibatch
    sigmas: (L,) tensor of noise levels (float)
    p:      (L,) optional sampling probs over noise levels (defaults uniform)
    """
    device = x.device
    B, D = x.shape
    L = sigmas.numel()

    # sample a noise level per example
    p = torch.full((L,), 1.0 / L, device=device)
    sigma_idx = torch.multinomial(p, num_samples=B, replacement=True) # (B,)
    sigma = sigmas[sigma_idx].view(B, 1) # (B,1)

    # corrupt data
    z = torch.randn_like(x)
    x_tilde = x + sigma * z # (B,D)

    # model score prediction
    s_pred = score_model(x_tilde, sigma_idx) # (B,D)
    # DSM target score for Gaussian corruption
    s_target = -(sigma * z) / (sigma ** 2) # (B,D) == -z/sigma
    # per-sample squared error
    err2 = (s_pred - s_target).pow(2).sum(dim=1) # (B,)

    # common weighting: multiply by sigma^2 (NCSN-style) to equalize scales
    if weight_by_sigma2:
        err2 = err2 * (sigma.view(-1) ** 2)

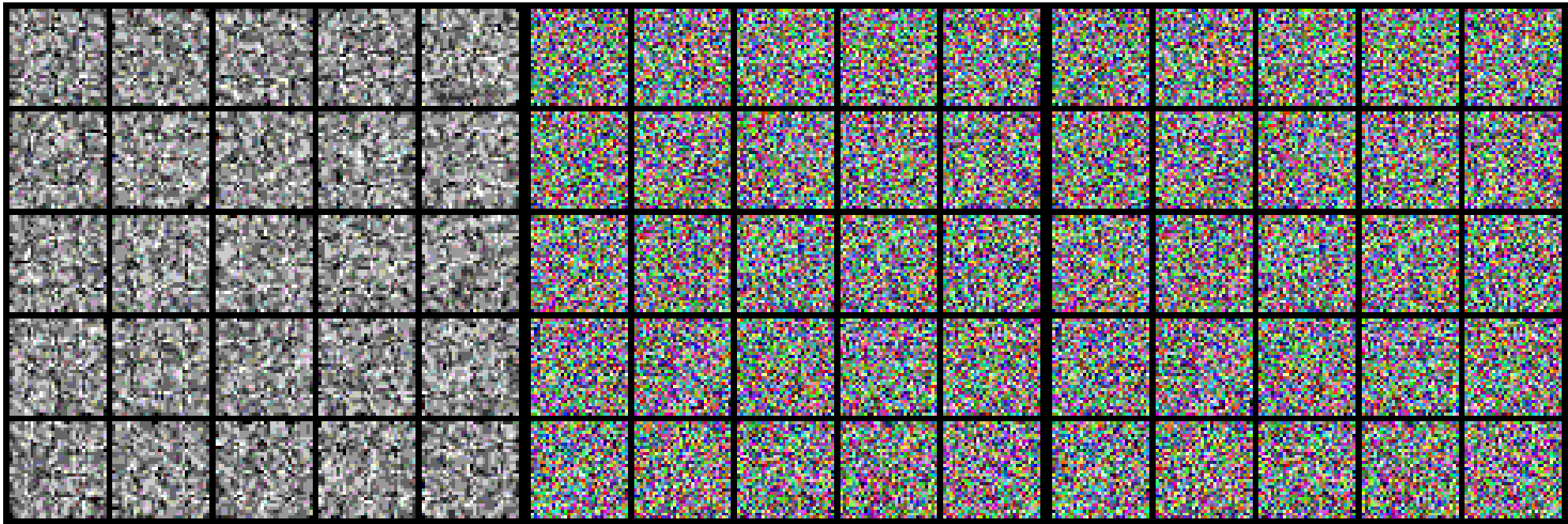
    return 0.5 * err2.mean()
```

Sampling

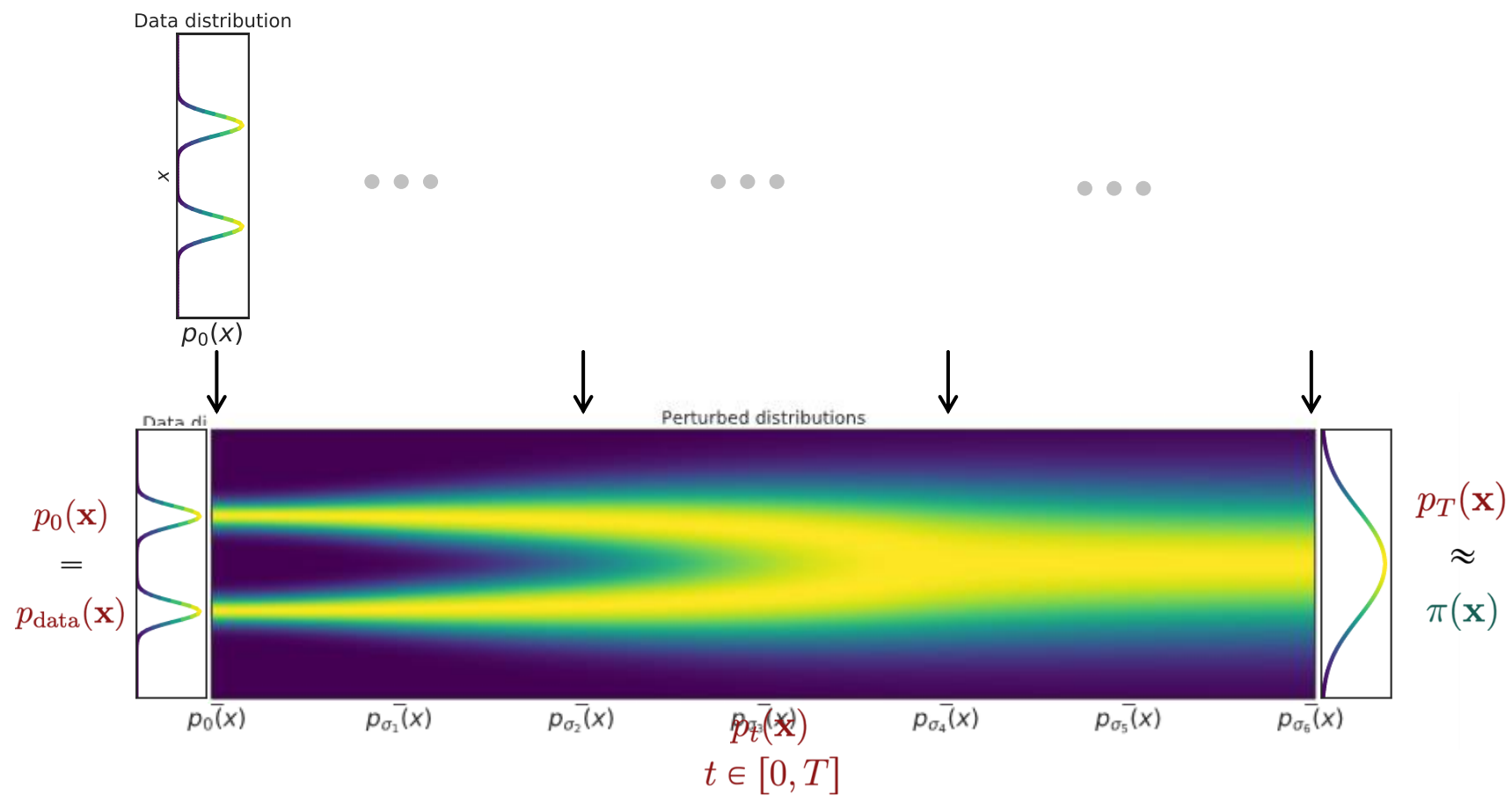
```
# -----  
# Geometric schedule helper (high -> low)  
# -----  
def geometric_sigmas(sigma_max, sigma_min, L, device=None):  
    """  
    Returns (L,) sigmas spaced geometrically from sigma_max down to sigma_min.  
    """  
    ratio = (sigma_min / sigma_max) ** (1.0 / (L - 1))  
    exponents = torch.arange(L, device=device, dtype=torch.float32)  
    return sigma_max * (ratio ** exponents)
```

```
@torch.no_grad()  
def annealed_langevin_geometric(  
    score_model,  
    batch_size: int,  
    dim: int,  
    n_steps_each: int = 50,  
    step_scale: float = 0.01,  
):  
    """  
    Assumptions:  
    - score_model(x, sigma_idx) -> score in R^D with same shape as x  
    - The model was trained with discrete noise levels (sigmas) (high -> low).  
    - Uses the common step size choice:  $\text{eps} = \text{step\_scale} * \text{sigma}^2$ .  
  
    Update (ULA at each noise level):  
    |  $x \leftarrow x + \text{eps} * s_{\text{theta}}(x, \text{sigma}) + \sqrt{2 * \text{eps}} * z$   
    |  
    """  
    # Initialize from  $N(0, \text{sigma\_max}^2 I)$   
    x = torch.randn(batch_size, dim, device=device) * sigmas[0]  
  
    for i, sigma in enumerate(sigmas):  
        sigma_idx = torch.full((batch_size,), i, device=device, dtype=torch.long)  
  
        eps = step_scale * (sigma.item() ** 2)  
        noise_std = (2.0 * eps) ** 0.5  
  
        for _ in range(n_steps_each):  
            s = score_model(x, sigma_idx) # (B,D)  
  
            z = torch.randn_like(x)  
  
            x = x + eps * s + noise_std * z  
  
    return x, sigmas
```

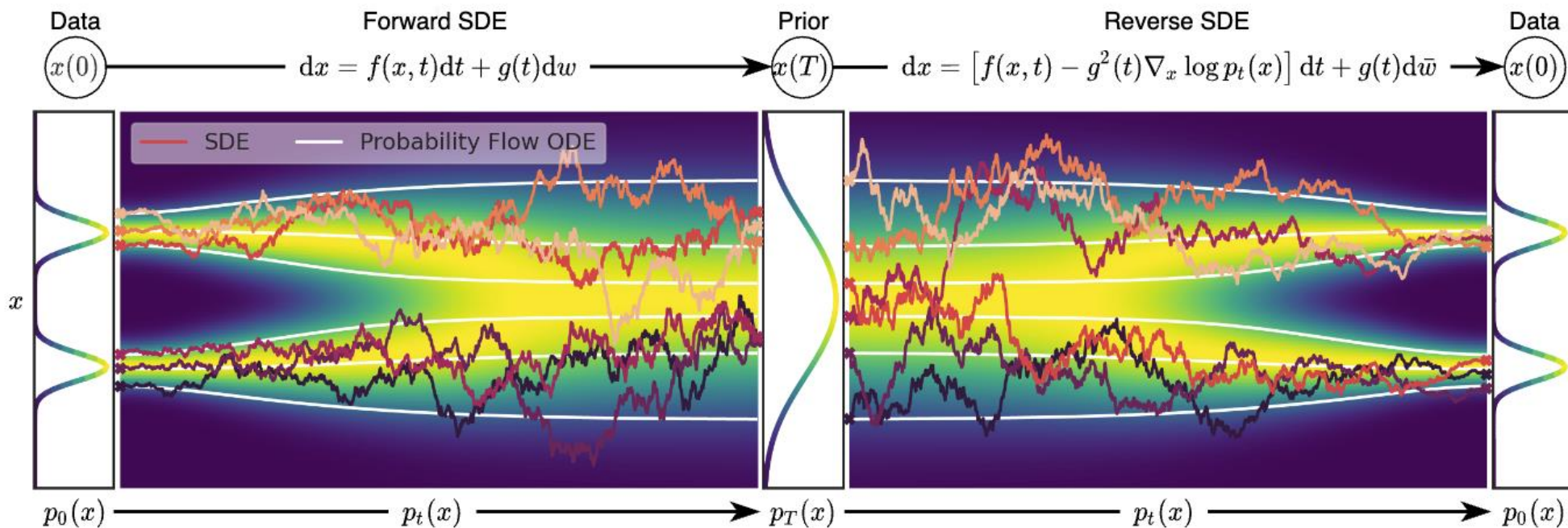
Sampling results



Continuous noise levels



Stochastic process



Stochastic process

$$\{\mathbf{x}_t\}_{t \in [0, T]}$$

Probability densities

$$\{p_t(\mathbf{x})\}_{t \in [0, T]}$$

Stochastic differential equation (SDE)

$$d\mathbf{x}_t = \underbrace{f(\mathbf{x}_t, t)}_{\text{Deterministic drift}} dt + g(t) \underbrace{d\mathbf{w}_t}_{\text{Infinitesimal noise}}$$

Deterministic drift

Infinitesimal noise

Simple SDE

$$d\mathbf{x}_t = \sigma(t) d\mathbf{w}_t$$

Reverse SDE ($t: T \rightarrow 0$)

$$d\mathbf{x}_t = -\sigma(t)^2 \underbrace{\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)}_{\text{Score function}} dt + \sigma(t) d\bar{\mathbf{w}}_t$$

Score function

Infinitesimal noise in the reverse time direction

Examples of SDE

- Score matching with Langevin dynamics (SMLD)

Discrete Markov chain $\mathbf{x}_i = \mathbf{x}_{i-1} + \sqrt{\sigma_i^2 - \sigma_{i-1}^2} \mathbf{z}_{i-1}, \quad i = 1, \dots, N, \quad \mathbf{z}_{i-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

Continuous SDE $d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w}$ Variance Exploding (VE) SDE

- Denoising diffusion probabilistic modeling (DDPM)

Discrete Markov chain $\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}, \quad i = 1, \dots, N.$

Continuous SDE $d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}.$ Variance Preserving (VP) SDE

Score-based diffusion models

- Examples
 - Score matching with Langevin dynamics (SMLD)
 - Denoising diffusion probabilistic modeling (DDPM)

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)(1 - e^{-2\int_0^t \beta(s)ds})}d\mathbf{w}.$$

- Training
 - Same as score matching
- Sampling
 - Euler–Maruyama discretization

Choose a decreasing time grid:

$$t_0 = T > t_1 > \dots > t_N = \varepsilon,$$

with step size $\Delta t_i = t_i - t_{i+1} > 0$,

Euler–Maruyama update for the reverse SDE:

$$x_{i+1} = x_i + (f(x_i, t_i) - g(t_i)^2 s_\theta(x_i, t_i)) \Delta t_i + g(t_i) \sqrt{\Delta t_i} z_i \quad \text{where } z_i \sim \mathcal{N}(0, I) \text{ i.i.d.}$$

Reverse drift

Stochastic
noise injection

Euler–Maruyama sampler

Inputs

- Forward SDE coefficients $f(x, t), g(t)$
- Score model $s_\theta(x, t) \approx \nabla_x \log p_t(x)$
- Number of steps N , end time T , small ε
- Prior sampler for $x^{(T)}$ (commonly $x^{(T)} \sim \mathcal{N}(0, I)$ for VP/VE SDEs)

1. **Initialize** $x_0 \sim p_T$ (e.g., standard Gaussian).

2. **Create time grid** $t_0 = T, t_1, \dots, t_N = \varepsilon$.

3. **For** $i = 0, 1, \dots, N - 1$:

- **Set** $\Delta t = t_i - t_{i+1}$.

- **Sample** $z \sim \mathcal{N}(0, I)$.

- **Compute reverse drift:**

$$\mu = f(x_i, t_i) - g(t_i)^2 s_\theta(x_i, t_i).$$

- **Euler–Maruyama step:**

$$x_{i+1} = x_i + \mu \Delta t + g(t_i) \sqrt{\Delta t} z.$$

4. **Output** x_N as a sample from (approximately) p_0 .